

Advanced Reading in Computer Vision (MAT3563)

LAB 04. TWO-STAGES OBJECT DETECTION MODELS

Trong bài thực hành này, chúng ta tìm hiểu một cách tương đối đầy đủ về 2-stage object detection xoay quanh Faster R-CNN. Mục tiêu của bài học này là:

- (1) Hiểu từng module (FPN, RPN, RoI Pool/Align, NMS, box/class heads) qua các bài mini-lab có ví dụ trực quan;
- (2) Chạy suy luận/dự đoán với mô hình pretrained;
- (3) Fine-tune trên dữ liệu mà chúng ta tự xây dựng (theo chuẩn Pascal VOC hoặc Ms. COCO) — cả với PyTorch (torchvision) lẫn TensorFlow (TF-Hub để suy luận/dự đoán nhanh + TensorFlow Object Detection API để fine-tune chuẩn). Sau bài thực hành này, học viên cần làm chủ được việc tinh chỉnh/huấn luyện lại mô hình Faster R-CNN trên một tập dữ liệu mới (có thể khác về tập nhãn, khác về miền nội dung ảnh – ví dụ ảnh y tế...).

1. MỤC TIÊU BÀI THỰC HÀNH & YÊU CẦU MÔI TRƯỜNG

Các bạn cần nắm được:

- Kiến trúc Faster R-CNN và luồng dữ liệu qua FPN → RPN → RoI Align → Heads → NMS.
- Thực hành từng “mắt xích” bằng các script nhỏ, nhìn thấy đầu-vào/đầu-ra.
- Chạy **pretrained inference** (với cả thư viện Torch & TF).
- **Fine-tune** trên dữ liệu Pascal VOC-style (XML) hoặc COCO-style (json).

Môi trường gợi ý:

- Python ≥ 3.9, CUDA nếu có.
- PyTorch ≥ 2.2, torchvision ≥ 0.17.
- TensorFlow ≥ 2.12 (GPU nếu có).
- (Tuỳ chọn, để fine-tune TF) TensorFlow Object Detection API.

2. DỮ LIỆU & CẤU TRÚC THƯ MỤC MẪU

Bạn có thể dùng:

- **VOC-style**: dataset/Annotations/*.xml, dataset/JPEGImages/*.jpg, dataset/ImageSets/Main/train.txt, val.txt
- **COCO-style**: dataset/train/images/*.jpg, dataset/train/annotations.json, tương tự cho val.

Cấu trúc gợi ý (VOC):

```
project_root/
├── notebooks/                # bạn có thể đặt các notebook ở đây
├── modules_lab/              # mini-labs từng module
├── torch_frcnn/              # code PyTorch fine-tune
├── tf_frcnn/                 # code TF inference & fine-tune
├── dataset/
│   ├── JPEGImages/
│   ├── Annotations/
│   └── ImageSets/
│       └── Main/
│           ├── train.txt
│           └── val.txt
└── outputs/
```

Nếu bạn đã có VOC XML, ta có thể dùng luôn vì kiểu VOC dễ truy cập hơn. Nếu chưa, có thể nhanh chóng chuyển COCO→VOC hoặc dùng một tập nhỏ tự gán nhãn (LabelImg). Tập nguồn chương trình coco2voc.ipynb là công cụ chuyển đổi giữa 2 định dạng này. Các bạn có thể tham khảo.

Hướng dẫn sử dụng coco2voc.ipynb

(I) Mở notebook, chỉnh **User config**:

- COCO_JSON: đường dẫn tới instances_*.json
- IMAGES_DIR: thư mục chứa ảnh thực tế
- OUT_DIR: thư mục đầu ra kiểu VOC
- SET_NAME, GEN_IMAGE_SET, COPY_IMAGES, CATEGORIES (tuỳ chọn)

(II) Chạy các cell theo thứ tự.

(III) Kết quả:

```
OUT_DIR/
Annotations/*.xml
JPEGImages/*.jpg          # nếu COPY_IMAGES=True
ImageSets/Main/train.txt
# nếu GEN_IMAGE_SET=True và SET_NAME="train"
```

Mặc định, nếu **không mở được ảnh** (thiếu file/ảnh hỏng), mẫu đó sẽ **bị bỏ qua** để tránh sai kích thước.

3. MINI-LABS (HIỂU TỪNG MODULE QUA VÍ DỤ NHỎ)

Mỗi mini-lab là một file Python độc lập (chạy được trong notebook). Ảnh minh hoạ: bạn có thể dùng bất kỳ ảnh .jpg nào trong dataset/JPEGImages/ (toàn ảnh thực).

3.1 FPN – Feature Pyramid Network (trực quan hoá multi-scale)

Mục tiêu: thấy được vì sao cần multi-scale khi vật thể nhỏ/xa.

Ý tưởng: Tạo một backbone (ResNet50) trích xuất các feature map ở nhiều mức, gom lại thành pyramid {P3, P4, P5, ...}. Đoạn code dưới đây tạo ra các maps ở 3 mức phân giải khác nhau

```
# modules_lab/01_fpn_demo_pytorch.py
import torch, torchvision
from torchvision.models.feature_extraction import create_feature_extractor
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

img_path = "D:/QG_2022/new_data/QG3506.jpg" # thay bằng ảnh của bạn
img = Image.open(img_path).convert("RGB")

transform = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Resize((800, 800)), # để so kích thước
    torchvision.transforms.Normalize(mean=[0.485, 0.456, 0.406],
                                     std=[0.229, 0.224, 0.225]),
])

x = transform(img).unsqueeze(0)

# Lấy các feature từ ResNet50 (C3, C4, C5)
backbone = torchvision.models.resnet50(weights="IMAGENET1K_V2")
nodes = {"layer2": "C3", "layer3": "C4", "layer4": "C5"}
extractor = create_feature_extractor(backbone, return_nodes=nodes)
with torch.no_grad():
    feats = extractor(x)

for k, v in feats.items():
```

```
print(k, v.shape) # quan sát kích thước

# Gợi ý: thủ tục xây FPN: P5 = conv(C5), P4 = conv(C4) + upsample(P5), ...
```

Hiển thị các feature maps ở các level khác nhau

```
import numpy as np
import matplotlib.pyplot as plt

for k, v in feats.items():
    image_array = v[0, 10, :, :]

    plt.imshow(image_array, cmap='gray') # 'gray' colormap for grayscale images
    plt.colorbar() # Optional: Add a color bar to show intensity mapping
    plt.title("2D Array as Grayscale Image")
    plt.show()
```

Bài tập 1: Cho 1 ảnh đầu vào → xem kích thước mỗi pyramid level; nội suy (upsample) và hiển thị heatmap “ngẫu nhiên” để thấy vai trò multi-scale.

3.2 Anchors & RPN – sinh anchor, chấm điểm, lọc top-k

Mục tiêu: hiểu cơ chế phủ ảnh bằng anchors & vì sao RPN cần NMS để lọc.

Ý tưởng: RPN đặt lưới anchor (đa tỷ lệ, đa tỉ lệ khung) trên mỗi FPN level, cho ra objectness + bbox deltas.

Đoạn code dưới đây là ví dụ minh họa (dạng toy example) cho việc tạo ra các chuỗi anchor bounding boxes trên 01 ảnh đầu vào, với 03 scale khác nhau cũng như 03 ratio khác nhau (tất cả là 09 anchor bbox).

```
# modules_lab/02_rpn_anchor_demo.py
import torch
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

def generate_anchors(base_size=32, ratios=(0.5,1.0,2.0), scales=(8,16,32)):
    anchors = []
    for s in scales:
        area = (base_size * s) ** 2
        for r in ratios:
            w = np.sqrt(area / r)
            h = w * r
            anchors.append([-w/2, -h/2, w/2, h/2])
    return np.array(anchors, dtype=np.float32)

def shift_anchors(feat_h, feat_w, stride, base_anchors):
    shifts_x = np.arange(0, feat_w) * stride
    shifts_y = np.arange(0, feat_h) * stride
    shift_x, shift_y = np.meshgrid(shifts_x, shifts_y)
    shifts = np.stack([shift_x.ravel(), shift_y.ravel(),
                       shift_x.ravel(), shift_y.ravel()], axis=1)

    A = base_anchors.shape[0]
    K = shifts.shape[0]
    all_anchors = (base_anchors.reshape((1,A,4))
                   + shifts.reshape((K,1,4))).reshape(-1,4)

    return all_anchors

# ví dụ: feature map 50x50, stride=16 → ảnh ~800x800
base_anchors = generate_anchors()
all_anchors = shift_anchors(50, 50, 16, base_anchors)
print("Anchors:", all_anchors.shape) # ~ 50*50*len(ratios)*len(scales)

# Vẽ một số anchor quanh tâm
center = (25, 25)
idxs = []
```

```

for dy in range(-1,2):
    for dx in range(-1,2):
        y = (center[0]+dy)*16; x=(center[1]+dx)*16
        # pick 3 anchors (1 scale)
        start = ((center[0]+dy)*50 + (center[1]+dx)) * base_anchors.shape[0]
        idxs.extend([start, start+1, start+2])

img_path = "D:/QG_2022/new_data/QG3506.jpg" # thay bằng ảnh của bạn
img = Image.open(img_path).convert("RGB")
img = img.resize((800, 800))

plt.imshow(img); ax = plt.gca()
for i in idxs:
    x1,y1,x2,y2 = all_anchors[i]
    rect = plt.Rectangle((x1,y1), x2-x1, y2-y1, fill=False, linewidth=1)
    ax.add_patch(rect)
plt.title("Local anchors around center cell")
plt.show()

```

Bài tập 2: Sinh anchor trên một level, cho các anchor trượt trên ảnh theo một stride tùy chọn, vẽ chồng lên ảnh để cảm nhận mật độ; kết hợp với phần 3.3 chạy **NMS** sơ bộ (ví dụ chỉ dựa vào IoU) để giữ ~100 proposals.

3.3 RoI Pool vs RoI Align – cắt/gộp đặc trưng chính xác phân số

Ý tưởng: RoI Pool dùng lượng tử hoá (rounded bins) → lệch; **RoI Align** dùng bilinear sampling không lượng tử → chính xác hơn.

Mục tiêu: Hiểu được cách để chuyển từ một vùng map có kích thước bất kỳ về cùng một cỡ chuẩn hóa.

PyTorch RoI Align (minh hoạ):

```

# modules_lab/03_roi_align_demo_pytorch.py
import torch, torchvision
from torchvision.ops import roi_align
from PIL import Image

# giả sử đã có feature map từ backbone (thay bằng output thực ở 01_fpn_demo)
feat = torch.randn(1, 256, 100, 100) # [N,C,H,W], ví dụ level P4

# Hộp trên ảnh gốc 800x800:
boxes_img = torch.tensor([[100,120,420,500],
                           [300,200,700,780]], dtype=torch.float32)

# roi_align kỳ vọng chuẩn hoá theo feature stride? Không cần: bạn truyền boxes theo
# toạ độ ảnh gốc + set spatial_scale.
spatial_scale = 100/800 # vì feat H,W=100 từ ảnh 800
rois = torch.cat([torch.zeros((boxes_img.size(0),1)), boxes_img], dim=1)
# prepend batch_idx
pooled = roi_align(feat, rois, output_size=(7,7), spatial_scale=spatial_scale,
                  aligned=True)
print(pooled.shape) # [num_rois, C, 7, 7]

```

Code trên Keras:

```

# modules_lab/03_roi_align_demo_tf.py
import tensorflow as tf
feat = tf.random.normal([1, 100, 100, 256]) # [N,H,W,C]
boxes_img = tf.constant([[100.,120.,420.,500.],
                          [300.,200.,700.,780.]])

# chuẩn hoá [y1,x1,y2,x2] về [0,1] theo ảnh gốc 800
norm = tf.constant([800.,800.,800.,800.])
y1x1y2x2 = tf.stack([boxes_img[:,1], boxes_img[:,0], boxes_img[:,3], boxes_img[:,2]],
                    axis=1) / norm
box_ind = tf.zeros((2,), dtype=tf.int32)

```

```
pooled = tf.image.crop_and_resize(feats, y1x1y2x2, box_ind, crop_size=(7,7),
method='bilinear')
print(pooled.shape) # [2,7,7,256]
```

Bài tập 3: Dùng torchvision.ops.roi_align hoặc tf.image.crop_and_resize để trích patch 7×7 từ feature map cho một box.

3.4 NMS – Non-Maximum Suppression (lọc chồng lấn)

Mục tiêu: Nắm được cách xử lý trong các bộ NMS.

Ví dụ minh họa: Tạo một nhóm box gần nhau với score khác nhau (Toy Example), chạy NMS và quan sát box giữ lại.

PyTorch:

```
# modules_lab/04_nms_demo_torch.py
import torch
from torchvision.ops import nms

boxes = torch.tensor([[50, 50, 200, 200],
                      [60, 60, 210, 210],
                      [400, 300, 600, 700]], dtype=torch.float32)
scores = torch.tensor([0.9, 0.8, 0.6])
keep = nms(boxes, scores, iou_threshold=0.5)
print(keep) # chỉ số box giữ lại
```

TensorFlow

```
# modules_lab/04_nms_demo_tf.py
import tensorflow as tf
boxes = tf.constant([[50., 50., 200., 200.],
                    [60., 60., 210., 210.],
                    [400., 300., 600., 700.]])
scores = tf.constant([0.9, 0.8, 0.6])
selected = tf.image.non_max_suppression(boxes, scores, max_output_size=10,
iou_threshold=0.5)
print(selected)
```

4. CHẠY MÔ HÌNH PRETRAINED (INFERENCE)

Trong phần này chúng ta chạy dự đoán mô hình Faster R-CNN với trọng số đã được pretrained trên tập ImageNet. Các bạn thay đường dẫn và file ảnh mà các bạn có, cũng như sửa đường dẫn đến thư mục chứa kết quả (outputs).

4.1 PyTorch (torchvision)

```
# notebooks/infer_torch_frcnn.ipynb (dùng như script .py cũng được)
import torch, torchvision
from PIL import Image, ImageDraw, ImageFont
import numpy as np

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model =
torchvision.models.detection.fasterrcnn_resnet50_fpn_v2(weights="DEFAULT").to(device).
eval()

transform = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor()
])

img_path = "dataset/JPEGImages/sample.jpg"
img = Image.open(img_path).convert("RGB")
x = transform(img).to(device)
with torch.no_grad():
    out = model([x])[0] # dict: boxes, labels, scores

boxes = out["boxes"].cpu().numpy()
```

```

labels = out["labels"].cpu().numpy()
scores = out["scores"].cpu().numpy()

draw = img.copy()
d = ImageDraw.Draw(draw)
for b,l,s in zip(boxes, labels, scores):
    if s < 0.5: continue
    x1,y1,x2,y2 = b
    d.rectangle([x1,y1,x2,y2], outline=(255,0,0), width=3)
    d.text((x1,y1), f"{int(l)}:{s:.2f}", fill=(255,0,0))
draw.save("outputs/torch_infer.jpg")
print("Saved outputs/torch_infer.jpg")

```

PHỤC LỤC. Sử dụng Torch API để xây dựng chương trình object detection Faster R-CNN

Chúng ta cũng có thể sử dụng Torch API để xây dựng chương trình object detection Faster R-CNN cho phép phát hiện các đối tượng có trong ảnh đầu vào, sử dụng trọng số đã được pretrained, phần training đã được thực hiện và tham số đã được lưu lại, bằng dữ liệu training của PyTorch. Thông tin chi tiết về tất cả các mô hình được pre-trained trong PyTorch có thể tìm thấy trong torchvision.models. Chương trình đầy đủ có trong tệp fasterRCNN1.ipynb đính kèm, ở đây ta chỉ giải thích một số đoạn lệnh.

Bước 1. Định dạng đầu vào và đầu ra

Mô hình pretrained Faster R-CNN ResNet-50 mà ta sẽ sử dụng cần tham số đầu vào có dạng [n, c, h, w], và có kích thước tối thiểu là 800px, trong đó:

- n là số hình ảnh
- c là số kênh, đối với hình ảnh RGB thì 3
- h là chiều cao của hình ảnh
- w là chiều rộng của hình ảnh

Mô hình sẽ trả về bộ giá trị

- Các **Bounding Box** được đánh dấu theo 2 góc [x0, y0, x1, y1] cho tất cả các lớp được dự đoán, tổ chức theo định dạng đầu ra là list có số chiều (N, 4) trong đó N là số lớp mà mô hình dự đoán sẽ có mặt trong hình, 4 chiều tiếp theo sẽ lần lượt tương ứng với 4 tọa độ của 2 góc trên trái và dưới phải ([x0, y0, x1, y1]).
- Nhân (tên đối tượng) của tất cả các lớp được dự đoán.
- Xác suất đối tượng khớp với từng nhãn được dự đoán.

Bước 2. Pretrained Model

Ta cần khai báo các thư viện cần thiết và khởi tạo mô hình Pre-Trained của Pytorch, đoạn lệnh khai báo thư viện sử dụng như sau:

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

import torchvision
from torchvision import transforms as T

```

Sau đó chúng ta khởi tạo mô hình Pre-Trained của Pytorch bằng đoạn lệnh sau:

```

model = torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrained=True)
model.eval()

```

Tiếp theo chúng ta khai báo tên một số phân loại đối tượng do Pytorch cung cấp. Ở đây một số phân lớp chúng ta sẽ không dùng tới nên tên phân lớp sẽ là 'N/A'

```

COCO_INSTANCE_CATEGORY_NAMES = [
    '_background_', 'person', 'bicycle', 'car',
    'motorcycle', 'airplane', 'bus', 'train', 'truck', 'boat', 'traffic light',
    'fire hydrant', 'N/A', 'stop sign', 'parking meter', 'bench', 'bird', 'cat',
    'dog', 'horse', 'sheep', 'cow', 'elephant', 'bear', 'zebra', 'giraffe', 'N/A',
    'backpack', 'umbrella', 'N/A', 'N/A', 'handbag', 'tie', 'suitcase', 'frisbee',
    'skis', 'snowboard', 'sports ball', 'kite', 'baseball bat', 'baseball glove',
    'skateboard', 'surfboard', 'tennis racket', 'bottle', 'N/A', 'wine glass',

```

```

'cup', 'fork', 'knife', 'spoon', 'bowl', 'banana', 'apple', 'sandwich',
'orange', 'broccoli', 'carrot', 'hot dog', 'pizza', 'donut', 'cake', 'chair',
'couch', 'potted plant', 'bed', 'N/A', 'dining table', 'N/A', 'N/A', 'toilet',
'N/A', 'tv', 'laptop', 'mouse', 'remote', 'keyboard', 'cellone', 'microwave',
'oven', 'toaster', 'sink', 'refrigerator', 'N/A', 'book', 'clock', 'vase',
'scissors', 'teddy bear', 'hair drier', 'toothbrush']

```

Bước 3. Sử dụng mô hình để chạy dự đoán/phát hiện đối tượng

Trong phần này chúng ta sẽ xây dựng phương thức `get_prediction(img_path, threshold)` trong đó `img_path` là đường dẫn đến file ảnh cần phát hiện đối tượng (test) và `threshold` là ngưỡng xác suất thuộc về một phân lớp mà chúng ta sẽ xác định cho đối tượng. Đầu ra sẽ là 04 tọa độ của Bounding Box và xác suất của Class.

```

def get_prediction(img_path, threshold):
    """
    get_prediction
    parameters:
        - img_path - path of the input image
        - threshold - threshold value for prediction score
    method:
        - Image is obtained from the image path
        - the image is converted to image tensor using PyTorch's Transforms
        - image is passed through the model to get the predictions
        - class, box coordinates are obtained, but only prediction score >
    threshold
        are chosen.
    """
    img = Image.open(img_path)
    transform = T.Compose([T.ToTensor()])
    img = transform(img)
    pred = model([img])
    pred_class = [COCO_INSTANCE_CATEGORY_NAMES[i] for i in
                  list(pred[0]['labels'].numpy())]
    pred_boxes = [[(i[0], i[1]), (i[2], i[3])] for i in
                  list(pred[0]['boxes'].detach().numpy())]
    pred_score = list(pred[0]['scores'].detach().numpy())
    pred_t = [pred_score.index(x) for x in pred_score if x>threshold][-1]
    pred_boxes = pred_boxes[:pred_t+1]
    pred_class = pred_class[:pred_t+1]
    return pred_boxes, pred_class

```

Tiếp theo ta xây dựng phương thức `object_detection_api(img_path, threshold=0.5, rect_th=3, text_size=3, text_th=3)` để gọi đến `get_prediction` và vẽ khung các đối tượng bằng thông tin được trả lại từ đó

```

def object_detection_api(img_path, threshold=0.5, rect_th=3, text_size=3, text_th=3):
    boxes, pred_cls = get_prediction(img_path, threshold)
    # Get predictions
    img = cv2.imread(img_path)
    # Read image with cv2
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    # Convert to RGB
    for i in range(len(boxes)):
        cv2.rectangle(img, boxes[i][0], boxes[i][1], color=(0, 255, 0),
                      thickness=rect_th)
        # Draw Rectangle with the coordinates
        cv2.putText(img, pred_cls[i], boxes[i][0], cv2.FONT_HERSHEY_SIMPLEX,
                    text_size, (0, 255, 0), thickness=text_th)
        # Write the prediction class
    plt.figure(figsize=(20, 30))

# display the output image
plt.imshow(img)
plt.xticks([])
plt.yticks([])
plt.show()

```

Bước dự đoán. Một số thực nghiệm trên ảnh

Chúng ta có thể lấy tập ảnh bất kỳ hoặc sử dụng các ảnh đính kèm, sau đó gọi thực hiện phương thức `object_detection_api` với các tham số tương ứng: `img_path`, `threshold` và kiểu chữ viết để có dự đoán. Chú ý thay đổi đường dẫn của file ảnh.

Trước hết chúng ta thực hiện với một số ảnh chỉ chứa đối tượng đã có trong list name của các class. Ví dụ:

```
object_detection_api('D:\\Teach_n_Train\\Advanced_Lessons_CV\\LABS\\faster-rcnn\\faces.jpg', threshold=0.8)
```

```
object_detection_api('D:\\Teach_n_Train\\Advanced_Lessons_CV\\LABS\\faster-rcnn\\traffic.jpg', rect_th=2, text_th=1, text_size=1)
```

```
object_detection_api('D:\\Teach_n_Train\\Advanced_Lessons_CV\\LABS\\faster-rcnn\\horse.jpg', rect_th=2, text_th=1, text_size=1)
```

```
object_detection_api('D:\\Teach_n_Train\\Advanced_Lessons_CV\\LABS\\faster-rcnn\\menhorses.jpg', rect_th=2, text_th=1, text_size=1)
```

Tiếp theo chúng ta thử nghiệm với một ảnh chứa các đối tượng không nằm trong danh sách tên các class đã biết bởi `pre_trained`.

```
object_detection_api('D:\\Teach_n_Train\\Advanced_Lessons_CV\\LABS\\faster-rcnn\\hobao.jpg', rect_th=2, text_th=1, text_size=1)
```

Phần code đầy đủ có thể tham khảo trong tệp nguồn đính kèm `fasterRCNN1.ipynb`.

4.2 TensorFlow (TF-Hub) – suy luận/dự đoán nhanh

TF-Hub có các Faster R-CNN pretrained (ResNet50/101). Ví dụ:

```
# notebooks/infer_tfhub_frcnn.ipynb
import tensorflow as tf
import tensorflow_hub as hub
import numpy as np
from PIL import Image, ImageDraw

# model URL ví dụ: "https://tfhub.dev/tensorflow/faster_rcnn/resnet152_v1_640x640/1"
MODEL_URL = "https://tfhub.dev/tensorflow/faster_rcnn/resnet50_v1_640x640/1"

detector = hub.load(MODEL_URL)

def load_img(path):
    img = Image.open(path).convert("RGB")
    return img, np.array(img)

img_path = "dataset/JPEGImages/sample.jpg"
pil, np_img = load_img(img_path)
input_tensor = tf.convert_to_tensor(np_img[None, ...]) # [1,H,W,3]

outputs = detector(input_tensor)
d_boxes = outputs["detection_boxes"][0].numpy() # [N,4] (y1,x1,y2,x2) in [0,1]
d_scores = outputs["detection_scores"][0].numpy()
d_classes = outputs["detection_classes"][0].numpy().astype(int)

H,W = np_img.shape[:2]
draw = pil.copy(); D = ImageDraw.Draw(draw)
for box,sc,cl in zip(d_boxes, d_scores, d_classes):
    if sc < 0.5: continue
    y1,x1,y2,x2 = box
    x1,y1,x2,y2 = x1*W, y1*H, x2*W, y2*H
    D.rectangle([x1,y1,x2,y2], outline=(0,255,0), width=3)
```

```
D.text((x1,y1), f"{cl}:{sc:.2f}", fill=(0,255,0))
draw.save("outputs/tfhub_infer.jpg")
print("Saved outputs/tfhub_infer.jpg")
```

5. FINE-TUNE (HUẤN LUYỆN LẠI / TÍNH CHỈNH)

5.1 PyTorch: fine-tune Faster R-CNN (VOC-style)

Bước A — Dataset (đọc VOC XML → boxes/labels): Như đã nói ở phần trước, các bạn có thể chuyển từ cách lưu thông tin kiểu Ms. COCO sang Pascal VOC (đã có tool), do đó mặc định dữ liệu phần này sẽ là VOC (tức là các file XML chứa thông tin các bounding box phục vụ huấn luyện. Nhắc lại, giả sử dữ liệu huấn luyện đã được tổ chức như ở phần 2.

```
# torch_frcnn/dataset_voc.py
import os, xml.etree.ElementTree as ET
from PIL import Image
import torch
from torch.utils.data import Dataset

class VOCDataset(Dataset):
    def __init__(self, root, image_set="train", classes=None, transforms=None):
        self.root = root
        self.transforms = transforms
        self.img_dir = os.path.join(root, "JPEGImages")
        self.ann_dir = os.path.join(root, "Annotations")
        ids_file = os.path.join(root, "ImageSets", "Main", f"{image_set}.txt")
        with open(ids_file) as f:
            self.ids = [x.strip() for x in f.readlines()]
        self.class_to_idx = {c:i+1 for i,c in enumerate(classes)} if classes else None

    def __len__(self): return len(self.ids)

    def _parse_xml(self, xml_path):
        tree = ET.parse(xml_path)
        root = tree.getroot()
        boxes, labels = [], []
        for obj in root.findall("object"):
            name = obj.find("name").text
            bnd = obj.find("bndbox")
            x1 = float(bnd.find("xmin").text)
            y1 = float(bnd.find("ymin").text)
            x2 = float(bnd.find("xmax").text)
            y2 = float(bnd.find("ymax").text)
            boxes.append([x1,y1,x2,y2])
            if self.class_to_idx:
                labels.append(self.class_to_idx[name])
            else:
                labels.append(1) # fallback 1 class
        return boxes, labels

    def __getitem__(self, i):
        img_id = self.ids[i]
        img_path = os.path.join(self.img_dir, f"{img_id}.jpg")
        ann_path = os.path.join(self.ann_dir, f"{img_id}.xml")
        img = Image.open(img_path).convert("RGB")
        boxes, labels = self._parse_xml(ann_path)
        boxes = torch.as_tensor(boxes, dtype=torch.float32)
        labels = torch.as_tensor(labels, dtype=torch.int64)
        target = {"boxes": boxes, "labels": labels, "image_id": torch.tensor([i])}
        if self.transforms:
            img = self.transforms(img)
        return img, target
```

Bước B — Xây dựng Model và train loop

Chú ý, trong phần này để tiết kiệm thời gian có thể đóng băng backbone một vài epoch đầu, hoặc đóng băng backbone chỉ sử dụng pretrained (code cell thứ 3 trong tập nguồn)

```
for name, p in model.backbone.body.named_parameters():
    p.requires_grad = False
# ... unfreeze dần theo epoch
```

Chương trình đầy đủ có thể tham khảo trong tập nguồn đính kèm `frncn_pytorch_finetune_voc.ipynb` . Dưới đây chúng ta chỉ xét một số đoạn code

```
# torch_frcnn/train.py
import torch, torchvision, os
from torch.utils.data import DataLoader
from dataset_voc import VOCDataset

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

classes = ["person", "car", "dog"] # ví dụ; đổi theo dataset
num_classes = len(classes) + 1    # + background

# Dataloader
transforms = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor()
])
train_set = VOCDataset(root="dataset", image_set="train",
                        classes=classes, transforms=transforms)
val_set    = VOCDataset(root="dataset", image_set="val",
                        classes=classes, transforms=transforms)
train_loader = DataLoader(train_set, batch_size=2, shuffle=True,
                           collate_fn=lambda b: tuple(zip(*b)))
val_loader   = DataLoader(val_set,   batch_size=2, shuffle=False,
                           collate_fn=lambda b: tuple(zip(*b)))

# Model
model = torchvision.models.detection.fasterrcnn_resnet50_fpn_v2(weights="DEFAULT")
in_features = model.roi_heads.box_predictor.cls_score.in_features
model.roi_heads.box_predictor =
torchvision.models.detection.faster_rcnn.FastRCNNPredictor(in_features, num_classes)
model = model.to(device)

# Optimizer
params = [p for p in model.parameters() if p.requires_grad]
optimizer = torch.optim.SGD(params, lr=0.005, momentum=0.9, weight_decay=1e-4)
lr_sched = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)

def to_device(batch):
    imgs, tgts = batch
    imgs = [im.to(device) for im in imgs]
    tgts = [{k:v.to(device) for k,v in t.items()} for t in tgts]
    return imgs, tgts

num_epochs = 10
for epoch in range(num_epochs):
    model.train()
    total_loss = 0.0
    for batch in train_loader:
        imgs, tgts = to_device(batch)
        loss_dict = model(imgs, tgts) # dict: classification, box_reg,
                                     # rpn_cls, rpn_reg
        loss = sum(loss_dict.values())
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
```

```

lr_sched.step()

# simple val (loss)
# torchvision detection expects train mode even for loss; for eval, do prediction
model.train()
val_loss=0.0
for batch in val_loader:
    imgs, tgts = to_device(batch)
    with torch.no_grad():
        ld = model(imgs, tgts)
        val_loss += sum(ld.values()).item()

print(f"Epoch {epoch+1}/{num_epochs} | train_loss={total_loss:.3f} |
      val_loss={val_loss:.3f}")

# save
os.makedirs("outputs", exist_ok=True)
torch.save(model.state_dict(), "outputs/frcnn_voc.pt")
print("Saved weights to outputs/frcnn_voc.pt")

```

Bước C — Suy luận sau fine-tune

Giống phần 4.1 nhưng tải các tên phân lớp đối tượng bằng phương thức **load state_dict** và sau đó đổi nhãn (labels) thành tên các classes của tập dữ liệu mà bạn đã xây dựng để vẽ label đúng.

5.2 TensorFlow: suy luận (TF-Hub) & fine-tune (TF OD API)

5.2.1 Suy luận/Dự đoán nhanh (đã có ở 4.2)

5.2.2 Fine-tune chuẩn với TensorFlow Object Detection API

Do thư viện TensorFlow xây dựng các layer và giao thức sử dụng chúng khá rõ ràng, có tính giải thích cao, nên cách dùng **TensorFlow Object Detection API** cho Faster R-CNN và thực hiện các bước tinh chỉnh trên đó là rất thích hợp. Tuy nhiên bù lại, việc setup sẽ dài và phức tạp hơn. Tóm tắt:

Trước hết cần thiết lập môi trường riêng nếu không muốn làm thay đổi quá nhiều các phiên bản đã cài đặt

Bước 1. Cài đặt (trong virtual-env/conda):

```

pip install tensorflow==2.13 tensorflow_io
pip install git+https://github.com/tensorflow/models.git#egg=object-
detection&subdirectory=research/object_detection
""" Hoặc clone repo 'models' và làm theo hướng dẫn cài protobuf nếu cần (Windows có
thể dùng precompiled .exe protoc)
"""

```

Bước 2. Chuẩn bị dữ liệu → TFRecord:

- Chuyển VOC XML → train.record, val.record (viết script chuyển đổi - Ở đây giả thiết dữ liệu training đang được để dưới dạng file kiểu PascalVOC).
- Tạo label_map.pbtxt (ví dụ):

```

item { id: 1 name: "person" }
item { id: 2 name: "car" }
item { id: 3 name: "dog" }

```

Ở đây cần đặt mapping tương ứng chỉ số của các nhãn đối tượng với tên của các đối tượng có trong tập dữ liệu mà bạn muốn huấn luyện lại. Ví dụ thay vì các tên đối tượng có trong tập ImageNet (1000 loại), bạn xây dựng tập mới với 3 loại đối tượng có tên như trên.

Bước 3. Chọn pipeline pipeline.config mẫu (Faster R-CNN + ResNet-50/101 + FPN). Sửa các trường:

- num_classes
- đường dẫn fine_tune_checkpoint (checkpoint pretrained)

- `train_input_reader` → `input_path`, `label_map_path`
- `eval_input_reader` → tương tự

Bước 4. Huấn luyện & đánh giá:

Chạy chương trình **model_main_tf2.py** sẽ có khi cài đặt tensorflow api object-detection đã đề cập ở bước 1. Các tham số cấu hình sẽ được đưa vào từ câu lệnh, từ đó các bạn có thể điều chỉnh lại cho phù hợp với hệ thống

```
python models/research/object_detection/model_main_tf2.py \
--model_dir=outputs/tf_frcnn_run \
--pipeline_config_path=tf_frcnn/pipeline.config \
--num_train_steps=20000 \
--sample_1_of_n_eval_examples=1
```

Bước 5. Xuất mô hình phục vụ infer:

Chạy chương trình **exporter_main_v2.py** sẽ có khi cài đặt tensorflow api object-detection đã đề cập ở bước 1. Các tham số cấu hình sẽ được đưa vào từ câu lệnh, từ đó các bạn có thể điều chỉnh lại cho phù hợp với hệ thống

```
python models/research/object_detection/exporter_main_v2.py \
--input_type=image_tensor \
--pipeline_config_path=tf_frcnn/pipeline.config \
--trained_checkpoint_dir=outputs/tf_frcnn_run \
--output_directory=outputs/tf_frcnn_exported
```

Chương trình đầy đủ tham khảo trong chương trình nguồn **tf_frcnn_finetune_odapi.ipynb**. Trong chương trình nguồn, chúng ta sẽ sử dụng một bộ dữ liệu nhỏ để làm minh họa (**PennFudanPed**). Các bạn có thể sửa lại để huấn luyện bằng bộ dữ liệu các bạn tự xây dựng. Có thể tóm tắt các bước thực hiện

- Cài đặt môi trường và TensorFlow Object Detection API
- Tải một tập dữ liệu nhỏ (**PennFudanPed**) và chuyển đổi sang định dạng TFRecord, đây sẽ được dùng làm tập huấn luyện
- Tạo tập **label_map.pbtxt** là tập chứa nhãn các đối tượng trong tập dữ liệu huấn luyện mới
- Tải **checkpoint** đã huấn luyện sẵn (**Faster R-CNN ResNet50 V1 FPN**) từ **TF2 Model Zoo**
- Tự động tạo file **pipeline.config** tương thích
- Huấn luyện mô hình bằng **model_main_tf2.py**
- Xuất **SavedModel** bằng **exporter_main_v2.py**
- Chạy **inference** và trực quan hóa kết quả detection

6. MỞ XE FASTER R-CNN VÀ THỰC HIỆN TỪNG CHỨC NĂNG

Hãy dựa vào các đoạn code cũng như phân tích đã có, hãy tách các module của mô hình Faster R-CNN sau đó thực hiện những công việc sau trên 01 ảnh thật.

- **Backbone + FPN**: trích xuất feature đa tỷ lệ (P3..P7), giúp phát hiện vật thể từ nhỏ → lớn.
- **RPN**: từ mỗi level, đặt anchors → conv nhỏ dự đoán **objectness** + **bbox delta**. Lấy top-k theo score, **NMS** để còn ~1–2k proposals.
- **Roi Align**: cắt patch 7×7 (hoặc 14×14) từ feature map tương ứng (chọn level theo kích thước box). **Align** (không lượng tử) > **Pool**.
- **ROI Heads**:
 - **Classification head**: per-Roi → phân lớp (softmax).
 - **Box regression head**: per-class (class-specific) hoặc class-agnostic → tinh chỉnh tọa độ box.
- **Post-process**: với dự đoán final (per class), áp **NMS** lần nữa → đầu ra[boxes, labels, scores].

7. BÀI TẬP NÂNG CAO & KIỂM THỬ

Thực hiện các yêu cầu sau

- **Ablations:** tắt FPN (chỉ dùng C5), so sánh độ chính xác vật thể nhỏ.
- **Roi Pool vs Align:** thay aligned=False (PyTorch) và đo ảnh hưởng.
- **Class-specific vs class-agnostic bbox head:** thử đổi predictor cuối (PyTorch).
- **NMS IoU sweep:** thử iou_threshold = {0.3, 0.5, 0.7} và quan sát số box & precision/recall.
- **Augmentations:** thêm random resize/flip/HSV... (torchvision transforms) và theo dõi loss.

8. CHEAT-SHEET (THAO TÁC NHANH/LÀM TẮT)

Torch – infer pretrained:

```
model =  
torchvision.models.detection.fasterrcnn_resnet50_fpn_v2(weights="DEFAULT").eval()  
out = model([tensor_img])[0]
```

Torch – fine-tune lớp cuối:

```
in_features = model.roi_heads.box_predictor.cls_score.in_features  
model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
```

Torch – NMS:

```
from torchvision.ops import nms  
keep = nms(boxes, scores, 0.5)
```

TF – infer TF-Hub:

```
detector = hub.load(MODEL_URL)  
outputs = detector(tf_img[None,...])
```

TF – NMS:

```
tf.image.non_max_suppression(boxes, scores, max_output_size=100,  
iou_threshold=0.5)
```

BÀI TẬP THỰC TẾ.

- Thu thập một tập dữ liệu khoảng 2000 ảnh về một lĩnh vực nào đó, sau đó thực hiện dự đoán bằng Faster R-CNN pretrained.
- Gộp các nhãn đối tượng để thu về một số nhóm đối tượng (ví dụ: car + truck = vehicles, all type of dogs = dog...) đặt lại thành các nhãn mới, sửa tệp đầu ra để thu được các tệp dạng VOC tương ứng.
- Sử dụng tập này như dữ liệu huấn luyện, tinh chỉnh hoặc huấn luyện lại mô hình.
- Thu thập khoảng 1000 mẫu ảnh mới của cùng lĩnh vực. Dùng mô hình đã huấn luyện để chạy dự đoán cho tập mới này.
- Tổng hợp kết quả.